GEN AI WITH LLMS · TOPIC 2

Transformers & LLMs

Attention, the Transformer Block, and Encoder Models

Laura Wynter, PhD
SMU School of Computing and Information Sciences
IS469 · AY2026–27

ROADMAP

What we'll cover

ngram solves text generation (system of chatbot) & TF-IDF/ BOW more for search engine. ngrams and TF-IDF are totally different tasks.

01 From RNNs/LSTMs to attention

02 The attention mechanism (Q, K, V)

03 The transformer block, end to end

04 Positional encoding & normalization

05 Modern efficient attention & FFN

06 The output head (decoding → Topic 5)

07 BERT, bidirectional encoder models

08 Multimodal LLMs

09 BERT embeddings, hands-on

10 Lab preview & wrap-up

FRAMING

Learning outcomes

- Explain the transformer architecture and how it powers today's LLMs
- Explain why transformers beat earlier sequence models (RNN, LSTM)
- Describe the attention mechanism and the query / key / value projections
- Understand what a transformer outputs when run on text
- Understand and use encoder models such as the BERT family

FRAMING

Transformers, in one slide

Transformer is type of NN underlying all GENAI

- A transformer is a type of neural-network architecture, introduced in 2017 in "Attention is All You Need" (Google)
- The original paper had both an encoder and a decoder; today most LLMs use the decoder only
- Attention fixes a core limit of static embeddings (e.g. word2vec): a word's meaning changes with its context
- Context is inferred from the surrounding words, attention computes that context-dependence efficiently
- This one idea scales: Llama-4, Qwen3, Gemma-3, GPT-5-class and Claude 4/5-class models are all transformers

01

From RNNs to Attention

Why the field abandoned recurrence, and what attention replaced it with.

RNN / LSTM vs. the transformer

NO NEED FOCUS FOR QUIZ

RNN / LSTM (sequential)

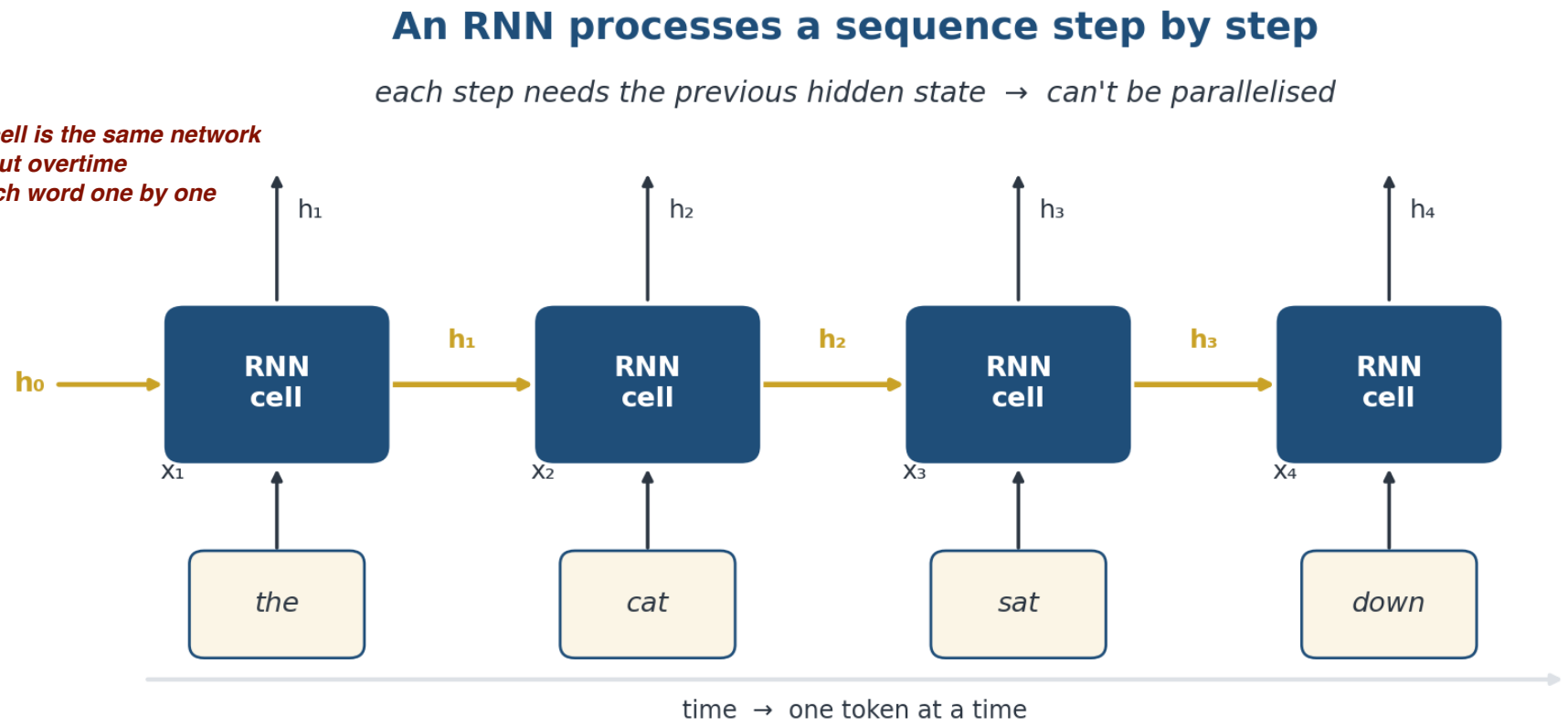
- Process a sequence one token at a time
- Current token + previous hidden state → next state
- LSTM gates control what information flows forward
- Limited parallelism, slow to train, hard to scale
- Still forget context from early in a long sequence

Transformer (parallel)

does not mean parallel processing

- Look at the whole sequence at once
- All tokens processed together, in parallel
- Connect every token to every other token
- Fast to train → scales to huge models & datasets
- Weighted sum focuses on the most relevant tokens

An RNN processes a sequence step by step



Each step consumes the current token and the previous hidden state, left to right, so it cannot be parallelised. The transformer removes this bottleneck by attending to all tokens at once.

What attention costs, and why it wins

- Attention lets the model connect each token to every other token, regardless of position
- At each step it computes a weighted sum of the sequence, focusing on the most relevant parts
- Because it operates on all pairs of tokens, cost grows quadratically with sequence length
- RNNs are linear in length but strictly sequential, attention is far faster in practice on modern hardware
- That quadratic cost is what modern efficiency tricks attack: sliding-window attention here in §5, and the inference-time KV-cache and batching in Topic 5

A worked example of attention

text generation - want to generate next word after it . We need model of all the words that came before it —> the model is attention mechanism

- "The chicken didn't cross the road because it …", what does "it" refer to?
- Self-attention weights for "it" are highest on "chicken", "it" attends most to "chicken"
- Attention rewrites a word's embedding using information from the surrounding words at the previous layer
- So the representation of "it" becomes context-dependent, the whole point attention set out to solve

02

The Attention Mechanism

Query, key, value, and the scaled dot-product at the heart of it.

Every token is three vectors

- Attention computes a weighted sum of the value vectors of other tokens, weighted by the current token
- Each token embedding is projected three ways, using matrices learned during training:

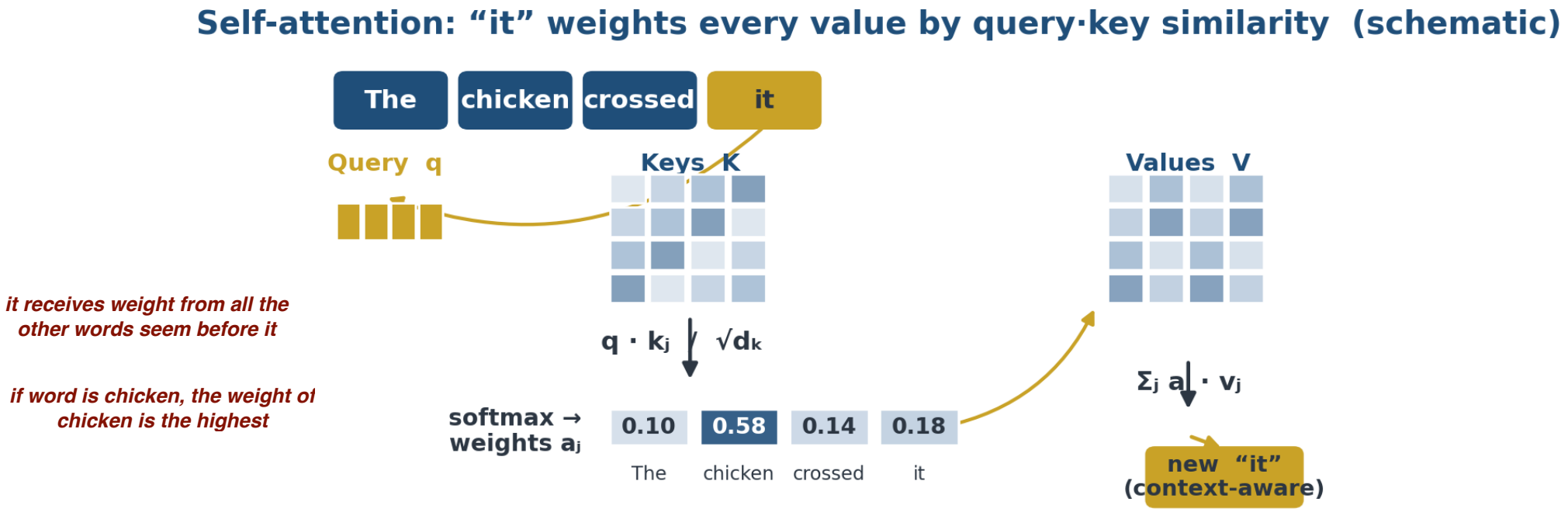
query is word u are acting on now (e.g it) *x is input embedding*

  - Query $q = xW_Q$, the current element doing the comparing (like a search "query")
  - Key $k = xW_K$, what each other token offers to be matched against (a key-value store)
  - Value $v = xW_V$, the content that gets weighted and summed (the data behind the key)
- W_Q, W_K, W_V are learned projections that reshape each input token into query / key / value

QKV are input embeddings modified by the weight matrix

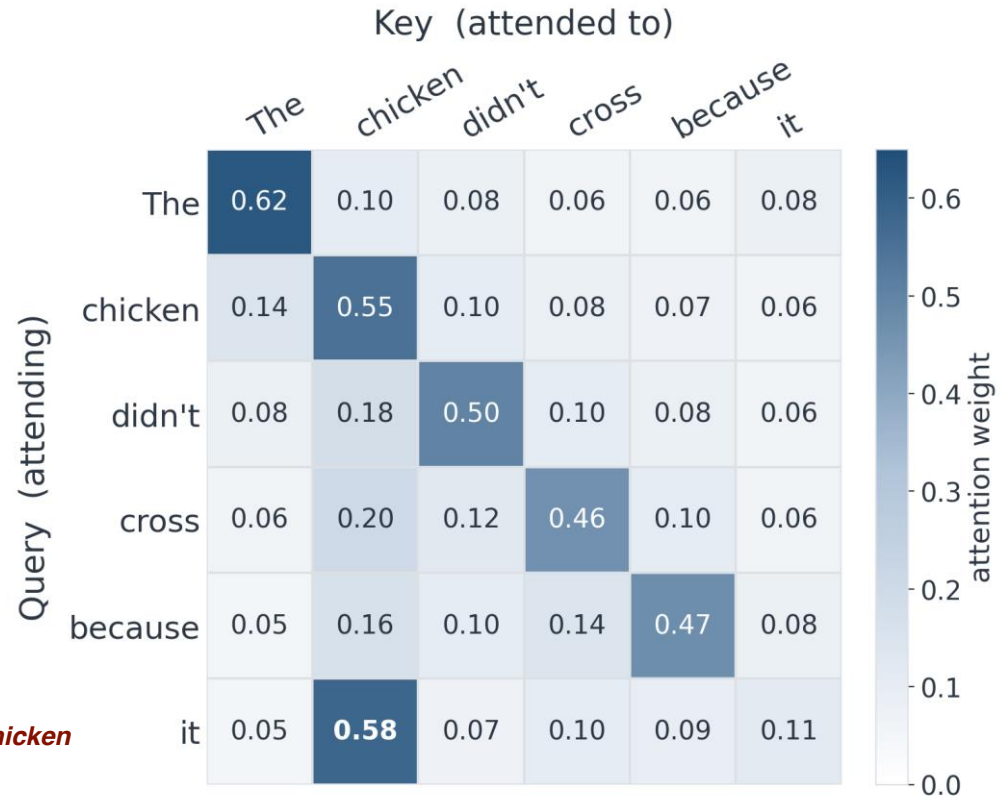
everytime u use a transformer, the input is modifying the embedding, taking into account all the words in input @ this point of time

How self-attention builds a context-aware token



The query for "it" is scored against every key; the softmax weights (highest on "chicken") blend the value vectors into a new, context-aware "it".

What attention actually learns



word will pay most attention to the word chicken

Each row is a query token's softmax over keys. The query "it" attends most to "chicken", resolving the reference.

how much attention should bank pay to river compared to itself?

§2 · WORKED EXAMPLE

embeddings changes as it propagates through the model

w only change when we are training the model, not when we are running the model, weights would have already been set to the optimal value

Attention with scalars: how W_k shifts focus

calculation happen at the first hidden layer. The input layer are the embedding of the tokens (x) but no longer one hot encoding like word2vec

K is everything we have seen so far

▸ We attend from the token "bank". Only the key weight W_K changes sign between the two cases — Q and V are ide

$$\text{Attention}(Q,K,V) = \text{softmax}\left(QK^\top/\sqrt{d_k}\right)V$$

$\text{softmax}(s_i) = e^{s_i}/\sum_j e^{s_j}$ — a bigger score gives a bigger weight; here $d_k = 1$, so the $\sqrt{d_k}$ scaling is skipped

SETUP

Inputs — normally embeddings are vectors here just using scalar
$x_{river} = 2,\ x_{bank} = 4$

Values ($W_V = 0.5$)
$v_{river} = 1,\ v_{bank} = 2$

Query ($W_Q = 3$)
$q_{bank} = 12$

4 (input vector of word acting on) x WQ

dimension is 1

Case 1 — $W_K = +2$

2x2=4
4x2=8

Keys: $k_{river} = 4,\ k_{bank} = 8$

score every token against the query

Scores $q_{bank} \times k$: river 48, bank 96 (12)

convert raw scores into weights that sums to 1

$\text{softmax}(48, 96) \rightarrow$ weights 0.01, 0.99

Vriver

Output $= 0.01(1) + 0.99(2)$

bank should pay 1% attention to the rvier and 99% attention to itself

= 1.99

captures bank's own value (2)

embedding of bank and river will change. This will go to next transformer block and it will change again

new embedding vector was 4 now it is 1.99 as it captures some information from river. modify embedding as a function of all the other words in the input

Case 2 — $W_K = -2$

Keys: $k_{river} = -4,\ k_{bank} = -8$

Scores $q_{bank} \times k$: river −48, bank −96

depending on words and weight, embedding is different. But for word2vec the matrix is fix , does not change with different words nor weights

$\text{softmax}(-48, -96) \rightarrow$ weights 0.99, 0.01

new embedding for bank is 1.01

Output $= 0.99(1) + 0.01(2)$

(−48 > −96, so river wins)

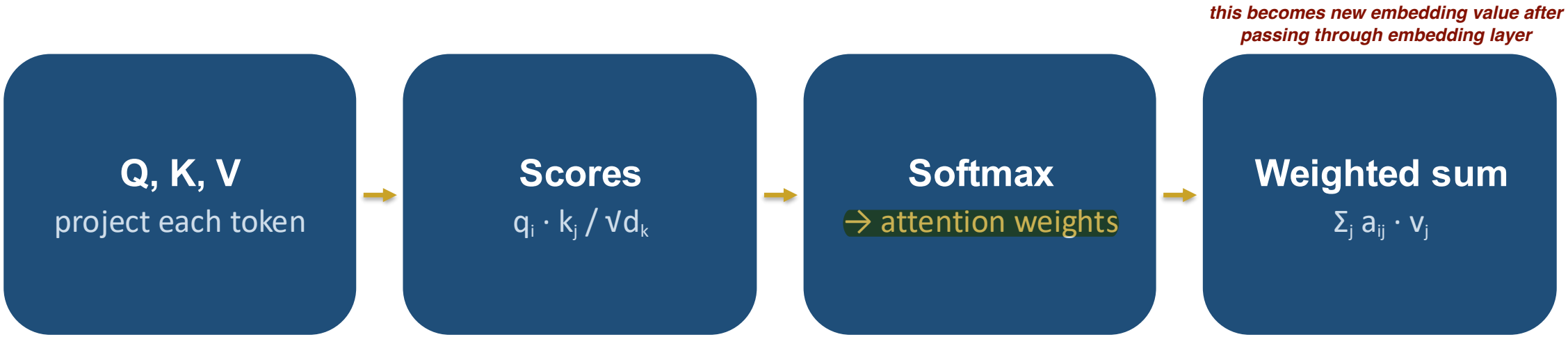
bank embedding becomes dominated by river's value

= 1.01

Same Q and V — only W_K flips sign. The key weights decide WHERE 'bank' attends.

embedding value changes. it propagates through model and changes as a function of other words in the input and weight matrix.

Self-attention, end to end



Each output token is a softmax-weighted blend of every value vector, $Attention(Q,K,V) = softmax(QK^T/\sqrt{d_k})V$.

Multi-head attention (MHA)

computing attention scores not once but N times in parallel, gives the model more perspective on text & meaning —> better representation of text

DRAWBACK: increase computational complexity/model becomes duplicative?

- One attention head captures one kind of relationship, we want many, in parallel
- MHA runs h heads, each with its own W_Q, W_K, W_V, attending to context with different weights
- "Attention is All You Need" used 8 heads; Llama-class models use 32+ heads
- Concatenate the heads, then multiply by an output projection to blend what each head learned
- Different heads specialise, syntax, coreference, position, without being told to

03

The Transformer Block, End to End

Stack the pieces: embedding, attention, FFN, norm, residuals, output head.

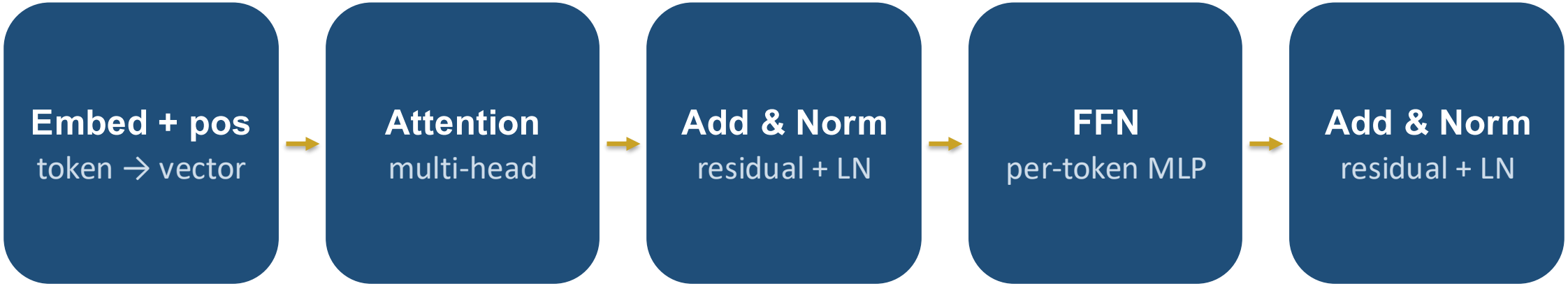
Inside one transformer block

transformed has multiple blocks stacks on each other, output from one block becomes input to another block

input to transformers are embedding

for each token, which token should it pay attention to?

adds a more non linearity to the calculation



Stack N identical blocks, then a linear prediction head + softmax → probabilities over the vocabulary.

Encoder and decoder, same block, different job

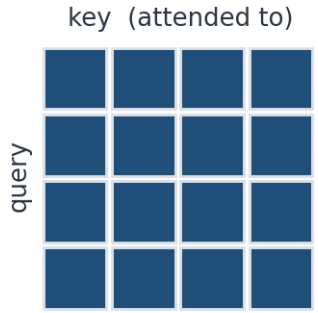
- The original "vanilla" transformer (built for machine translation) has an encoder and a decoder
- The blocks look almost identical, the differences are the inputs/outputs and the training loss
- Encoder: sees the full input, trained with masked-language-model (fill-in-the-blank) loss
- Decoder: can't see the future, trained with causal (next-token) loss
- Today's main LLMs are decoder-only, no encoder

One block, two attention masks

Same block, opposite masks: encoder attends everywhere, decoder only backward (schematic)

BERT, encoder

bidirectional · every token sees all others

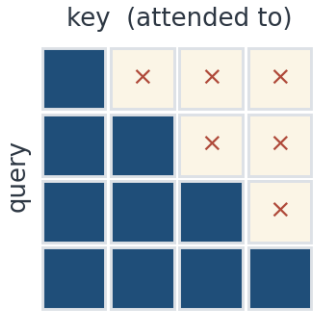


full attention → fill the blank from both sides



GPT, decoder

causal · each token sees only the past



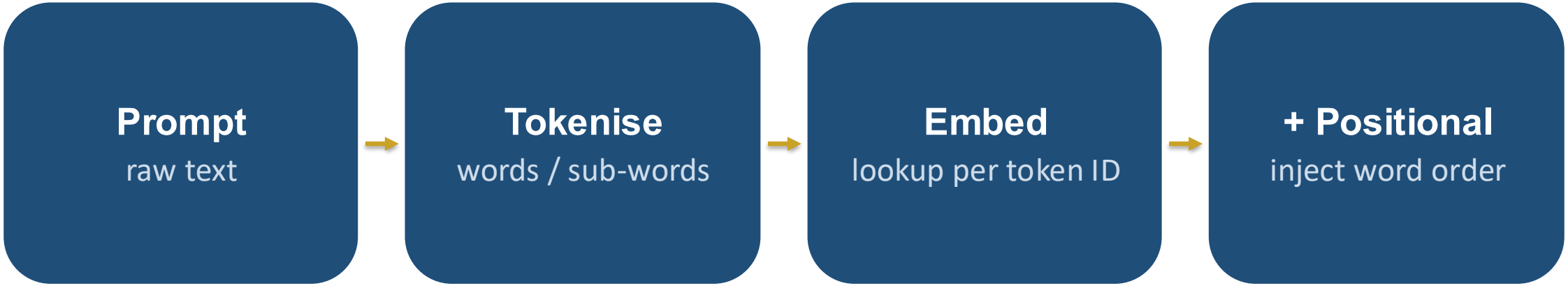
future masked (×) → predict the next token



BERT's encoder attends in every direction to fill a [MASK]; GPT's decoder masks the future (×) and predicts the next token left-to-right.

From text to the first layer

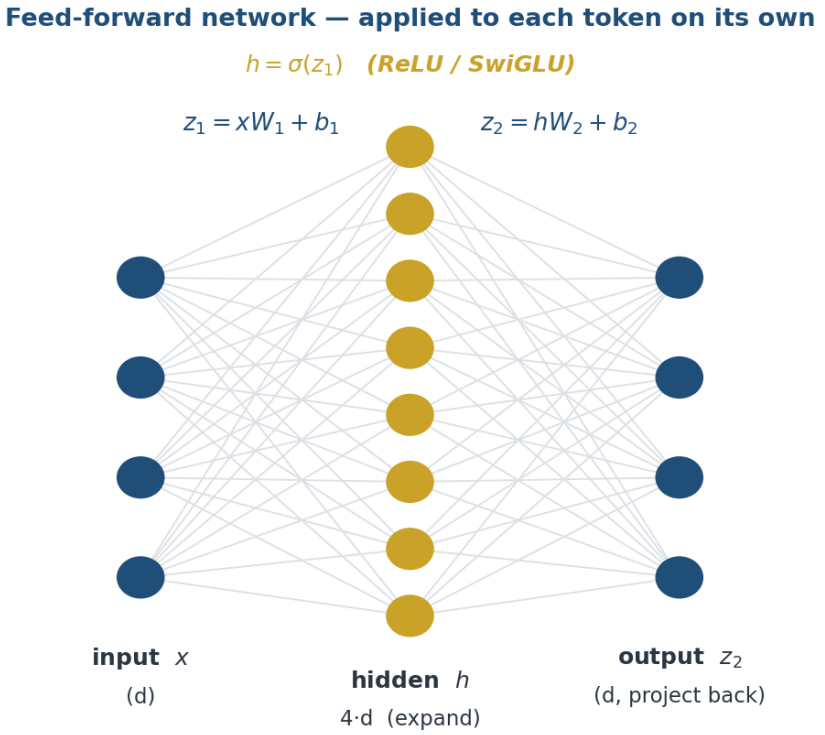
we get initial embedding for each of those tokens



Positional info: sinusoidal (original transformer), learned (BERT), or RoPE (modern decoders), see §4.

The feed-forward network (FFN)

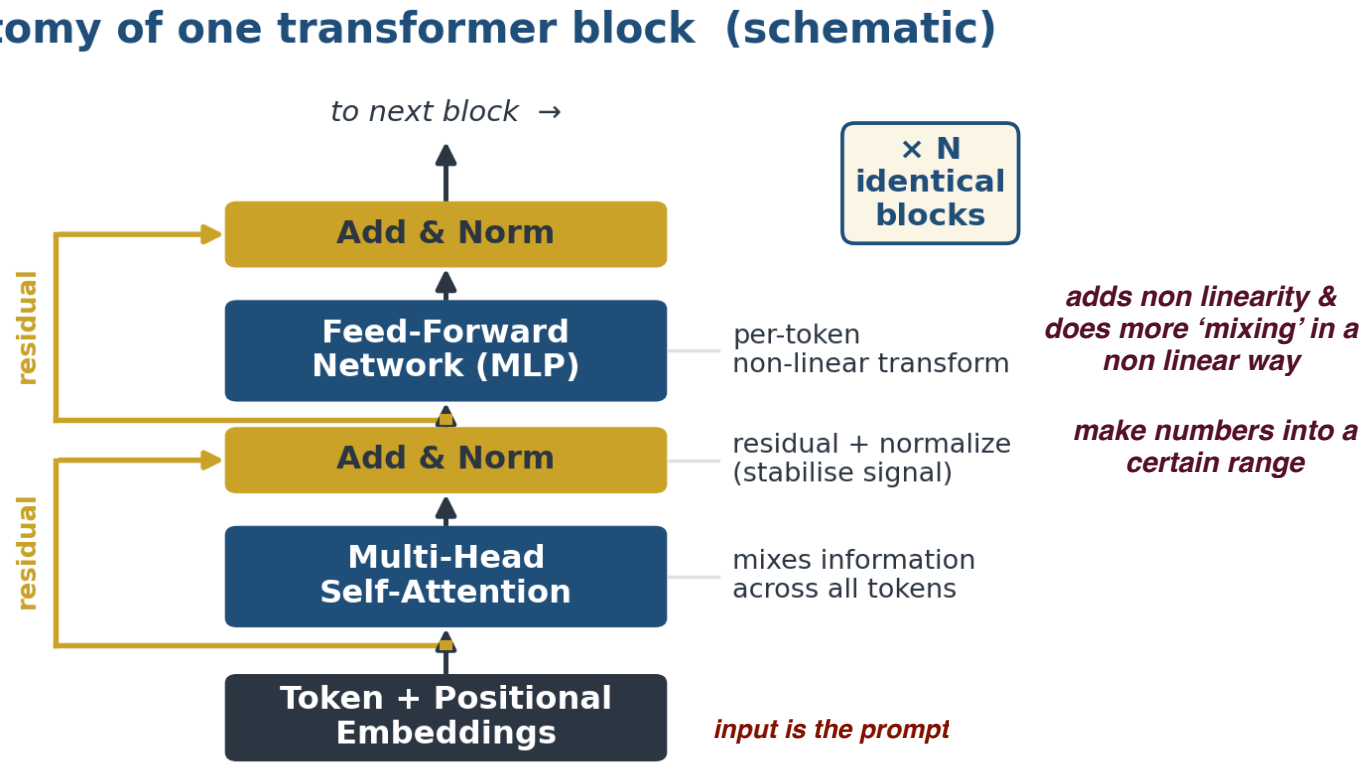
- After attention mixes tokens, the FFN (a.k.a. MLP) adds non-linearity to each position independently
- Two linear layers: expand from d to a much larger dim (e.g. 4096), then project back to d
- A non-linear activation sits between them:
  - $\mathrm{FFN}(x) = \mathrm{ReLU}(xW_1 + b_1)W_2 + b_2$
- The FFN typically holds the majority of a transformer's parameters



Residual connections

- Introduced in ResNet (vision CNNs), now standard in every transformer block
- Add a sub-layer's input to its output, information can bypass one or more layers
- Preserves signal and gives gradients a short path back through the network
- Directly mitigates vanishing gradients when back-propagating through many deep layers

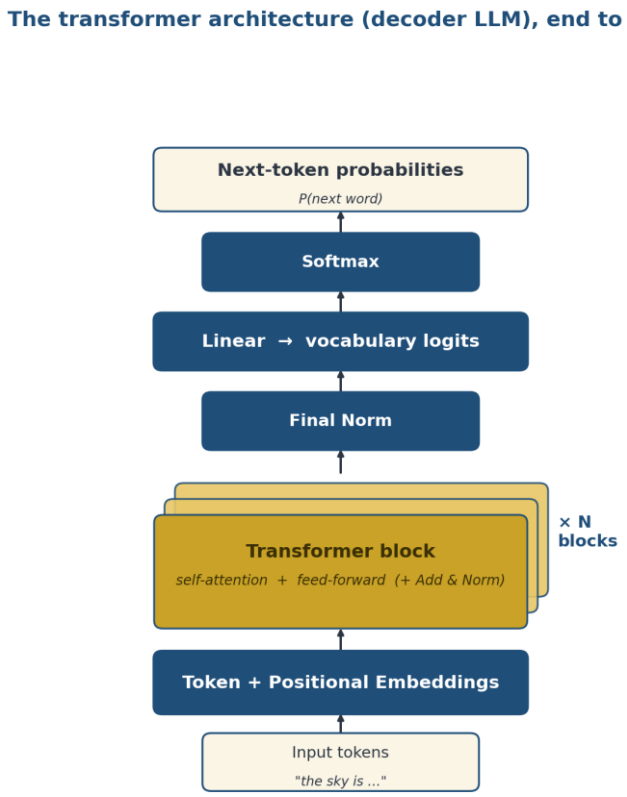
The whole block, in one schematic



Attention then FFN, each wrapped in a residual bypass (gold) + Add & Norm, the unit that stacks × N to make a transformer.

The transformer architecture, end to end

n.o of words in vocab is 500,000. For every token output is a probability distribution of length 500,000 just to select the next word.



today transformers have 100 billion of parameters/ weights (from attention and both fnn layer) to output the next word.

output is vector of probabilities that tells u the probability of the next word.

output layer is probability distribution over the entire vocab. Length of output layer is the length of vocab

The full decoder LLM: tokens → embeddings → a stack of N transformer blocks → final norm → linear → softmax → next-token probabilities.

04

Positional Encoding & Normalization

Two supporting mechanisms that make deep transformers trainable and order-aware.

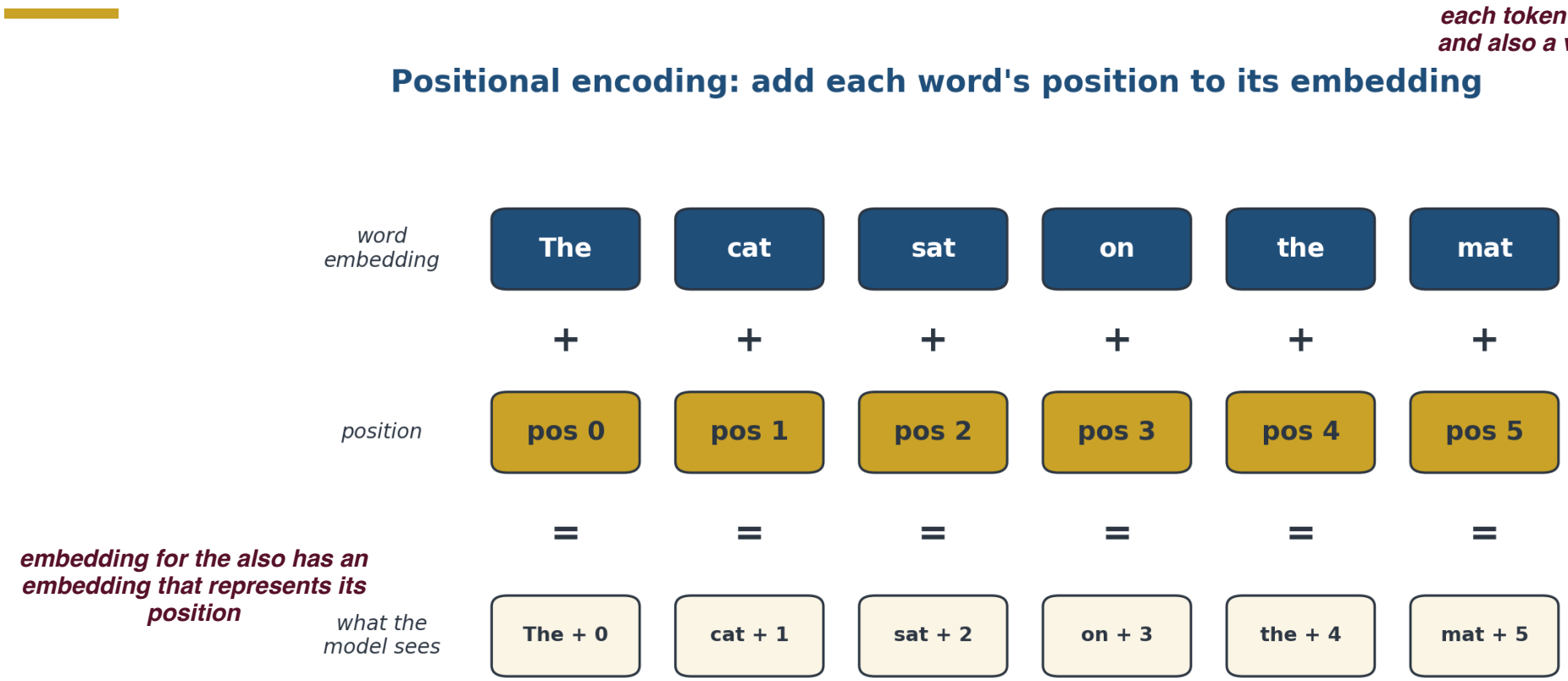
Sinusoidal encoding → RoPE

Attention calculate scores for all words unlike RNN @ same time hence it loses the order

- Attention alone is order-blind, positional encoding tells the model where each token sits
- Classic sinusoidal: a unique vector per position from sine/cosine in [−1, 1], added to the embedding
  - Bounded and the same size as the embedding, smoother than raw integer positions 1, 2, 3…
- RoPE (rotary positional encoding) rotates query/key vectors by an angle set by position
  - Encodes relative position, generalises to longer sequences, and is the default in modern decoders

The idea: stamp each word with its position

each token gets its own embedding and also a vector that represents its position



embedding for the also has an embedding that represents its position

Every word's embedding gets its position added in, so order-blind attention knows the sequence. "the" at 0 and at 4 become different vectors.

Layer norm, keeping activations sane

bring back the values into the same range so as to avoid numerical underflow or overflow

- During training, some weights and activations blow up → numerical instability
- Normalization rescales activations into a consistent range: subtract the mean, divide by the std
- Layer norm: $LN(x) = \gamma \cdot (x - \mu)/\sigma + \beta$, with learnable scale γ and shift β per feature
- Normalizes across the features of each token, independent of batch size and padding
- Transformers use layer norm, or the faster RMSNorm

§4 · AT A GLANCE

Normalization & position, what and where

fyi, not in quiz

Mechanism	What it does	Where used
LayerNorm	Center & scale across features	Original transformer, BERT
RMSNorm	Scale by RMS only (no mean)	Llama, Qwen, Gemma
Sinusoidal PE	Fixed sine/cosine, added up front	Original transformer
Learned PE	Learned position vectors, added	BERT
RoPE	Rotate Q/K by position (relative)	Modern decoders

Two supporting mechanisms: keep activations stable, and make an order-blind model order-aware.

05

Modern Efficient Attention & FFN

How 2026 models tame the quadratic cost and stretch context to millions of tokens.

Why efficiency matters, the big picture

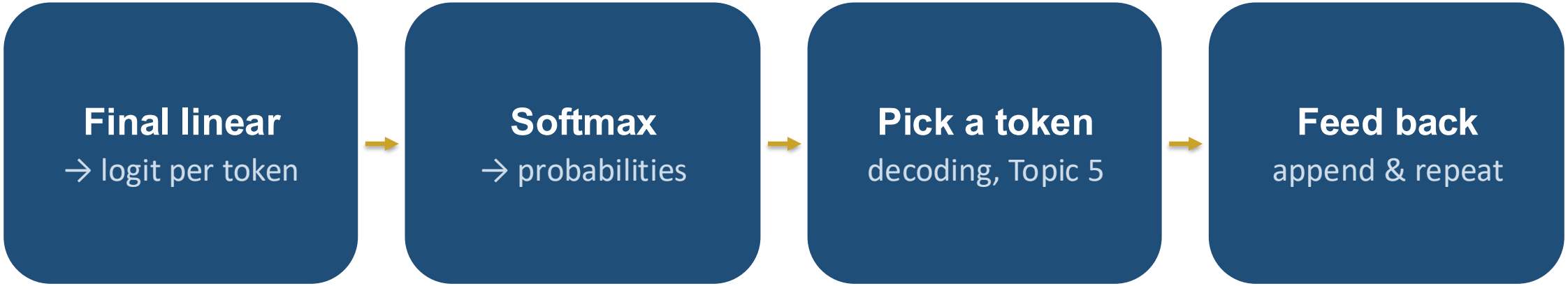
- Plain attention compares every token with every other, so work grows with the square of the sequence length
- That quadratic cost is what limits how long a context a model can handle
- This section covers the architectural answers that stretch context and capacity without blowing up compute:
  - Attend locally instead of to everything (sliding-window), and rescale positions to reach far past the training length (RoPE scaling)
  - Grow total capacity while keeping compute per token roughly flat (Mixture-of-Experts FFNs)
- Inference-time savings, the KV-cache, grouped-query attention and batching, come later in Topic 5 (Inference)

06

The Output Head

How processed embeddings become a probability distribution over the next token.

From embedding to the next-token distribution



The transformer outputs a probability distribution over the vocabulary. HOW you pick from it, greedy vs sampling, temperature, top-k, top-p, is a decode-time choice, covered in Topic 5 (Inference).

What the model actually outputs

- Run on text, a transformer produces one probability distribution over the vocabulary per position
- "Understanding the output" means reading that distribution: what does the model think comes next?
- Greedy / argmax just takes the top token; richer decoding shapes how much randomness you allow
- Those decoding controls (temperature, top-k, top-p) only apply at inference → we cover them in Topic 5, not here

07

BERT, Bidirectional Encoder Models

The encoder side of the family, still the workhorse for classification & embeddings.

BERT sees both directions

now LLMs use to predict next token the input that they use for the prediction, model look at the words up until now that it has seen. ONE DIRECTION (BACKWARDS).

BERT is different , use more for text representation. Give entire sentence to BERT and ask them for representation.

BERT is a TRANSFORMER MODEL

- Decoders (GPT-5-class, Claude 4/5-class…) run left-to-right, masking future tokens, autoregressive
- BERT is an encoder: bidirectional, it uses both the left and right context of each word
- "The bat flew ___ when the baseball player let it go from his hands."
  - A left-only decoder might read "bat" as the animal
  - BERT sees "baseball player" on the right too, and resolves "bat" correctly

BERT model train on words before and after (bidirecitonal)

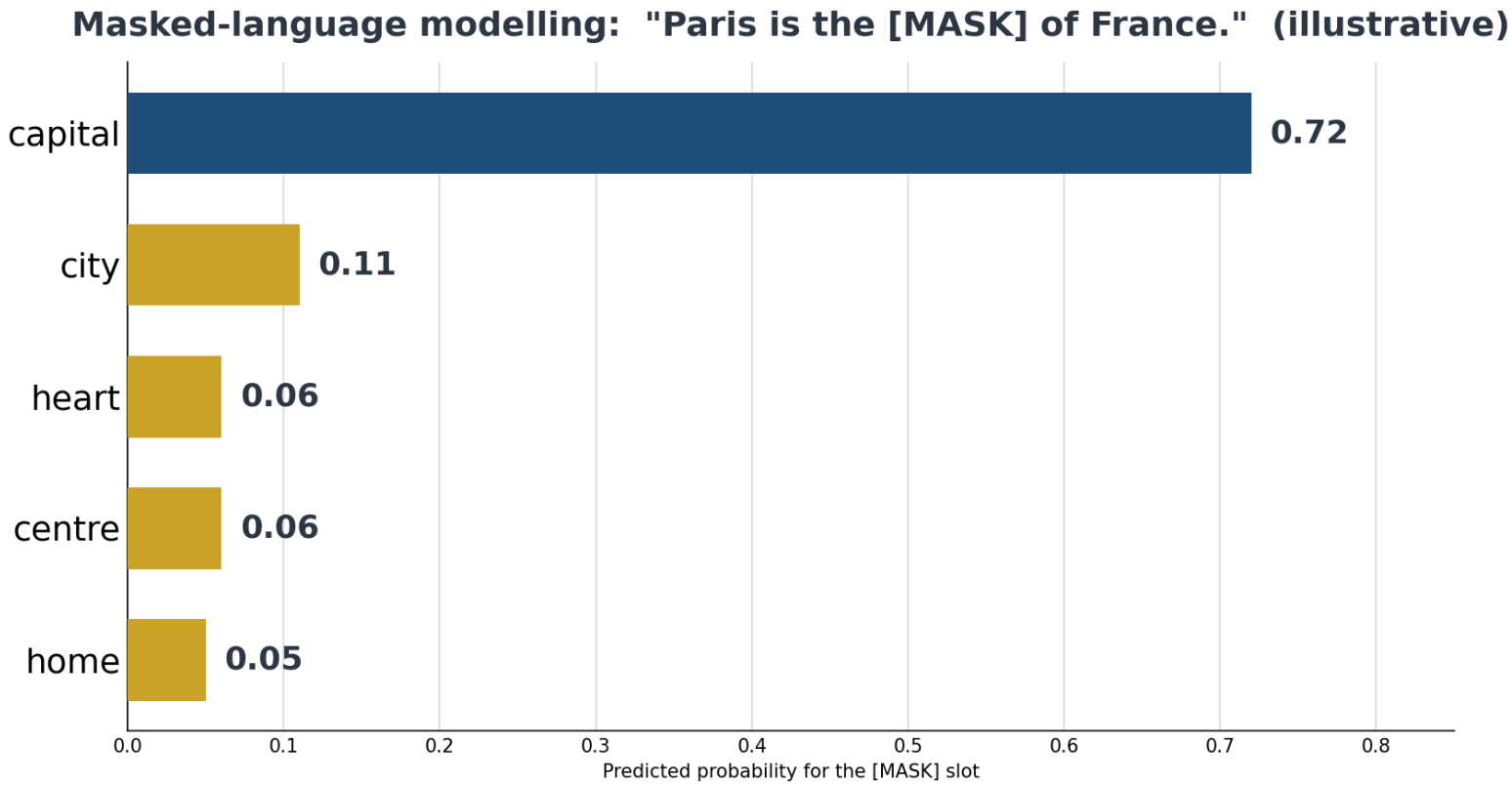
How BERT is trained

encoder- masking some text and predicting the text

current classification task we still use today: email filter, topic classification (processing review in the company)

- Trained with masked-language modeling (MLM): predict missing words from within a sentence
- Like word2vec's CBOW in spirit, but with a full transformer, not a 2-layer network
- BERT-base: 12 layers × 12 heads, 110M params (≈ GPT-1's 117M, also 2018)
- BERT-large: 24 layers × 16 heads, 340M params
- Encoder-only → built to understand input sequences, not to generate long text

Masked-language modelling, in one picture



BERT predicts a masked token from both-side context, here left ("Paris is the") and right ("of France") both point to "capital".

Why BERT still matters in 2026

- An early model, but far smaller and cheaper than frontier decoders, and it doesn't need to generate
- Text classification via a fine-tuning dataset, sentiment, topic, intent, spam, moderation…
- Embedding generation, feed the vectors into clustering, search or downstream ML
- The pattern lives on: modern sentence-embedding models are encoder descendants of BERT

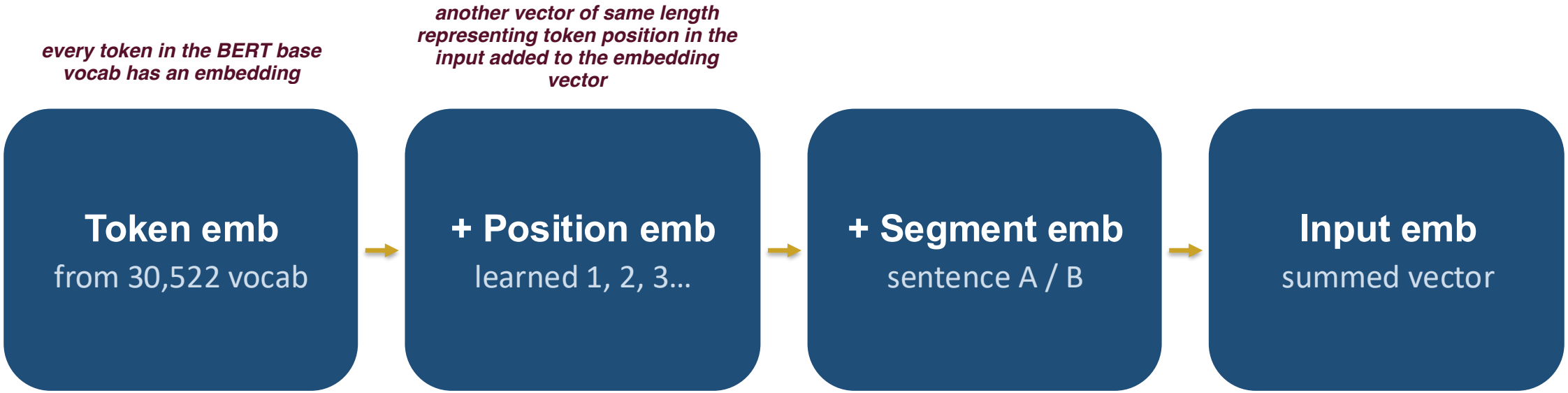
BERT's embedding layer

- BERT sums three embeddings per token: (1) token, (2) position, (3) token-type (segment)
- Contrast with RoPE in recent decoders, where position is applied inside attention, not added up front

mark beginning

- Special tokens: [CLS] at the front, [SEP] between/after segments
- "hello, world!" → IDs 101 7592 1010 2088 999 102 ([CLS] … [SEP])
- Token embeddings are learned over BERT's 30,522-word vocabulary

BERT sums three embeddings per token



Contrast with RoPE decoders, where position is applied inside attention, not added up front.

Using BERT: the [CLS] token

- The embedding layer output has shape (batch, num_tokens, hidden), e.g. (1, 6, 768)
- [CLS] aggregates information from the entire sequence, a pooled representation
- Feed [CLS] into a classification head to make predictions for many tasks
- Alternative pooling: mean-pool the token embeddings, we'll compare both in the lab

08

Multimodal LLMs

When the input isn't only text, encoders and decoders working together.

Encoders + decoders for images and text

- From Llama 3.2 onward, and now Llama-4 / Gemma-3 / GPT-5-class, LLMs are natively multimodal
- Prompt with an image plus text, "How many dogs are in this image?", and get text (or images) back
- An encoder turns the image into embeddings; a decoder consumes the combined image+text input
- So the encoder/decoder pieces from §3 recombine, encoders didn't disappear, they moved into the vision path

09

Embeddings, Hands-On

Static vs. contextual, what you get with and without a forward pass.

Static vs. contextual embeddings

- Read vectors straight from the embedding layer → static embeddings (one vector per vocab token)
- These are not context-dependent, "bank" is the same vector in every sentence
- Run a full forward pass through the model → contextual embeddings that shift with surrounding words
- Compare word similarity with cosine similarity: $\cos(x, y) = (x \cdot y) / (|x|\ |y|)$
- In the lab we'll measure both, and watch context change the answer

Key takeaways

01

Attention replaced recurrence

Processing all tokens in parallel with Q/K/V scaled dot-product attention is what let LLMs scale.

02

One block, stacked deep

Embedding + positional encoding, then MHA + FFN wrapped in residuals and normalization, then a softmax head.

03

Modern efficiency is the real story

Sliding-window attention, RoPE scaling and MoE stretch context and capacity here; KV-cache, GQA/MQA and batching come in Topic 5 (Inference).

04

Encoders still earn their keep

BERT-family encoders remain the cheap, strong choice for classification and embeddings.

LAB

Hands-on lab & what's expected

- Topic2.ipynb, BERT hands-on: tokenizer, embedding layer, cosine similarity, and [CLS] vs mean pooling
- See static vs. contextual embeddings by comparing pre- and post-forward-pass vectors
- BERT stays a small, pedagogical stand-in for the frontier models we've named